SOFTWARE TESTING

Practical Learning Guide

10 modules • real examples • test design • practice assessment

How to use this guide

Study the explanation, walk through the example, and try the knowledge checks before reading the answers. Use the practice assessment to identify gaps and revise the relevant module. The goal is to understand how to choose, design, execute and report tests in realistic situations.

Assessment target: 70%. This guide includes a 20-question practice assessment; 14 correct answers reaches the target.

Module 1 — Introduction to Software Quality

What quality means and who is responsible for it

Learning objectives

Explain verification, validation, the testing life cycle and shift-left quality.

Quality and the testing life cycle

Software quality means the product behaves as required, is useful to its users, and has acceptable reliability, security, performance and usability. Testing is not only a final activity. A common testing life cycle includes requirement analysis, test planning, test design, environment and data preparation, execution, defect reporting, retesting, regression testing and closure/reporting. Teams may iterate through these activities in each sprint.

Verification versus validation

Verification asks, “Are we building the product according to the specification?” Examples include reviewing requirements, inspecting code, checking a design against standards and running tests against defined acceptance criteria. Validation asks, “Are we building the right product for the user's real need?” Examples include usability sessions, acceptance testing and checking whether a workflow solves the intended problem. Both are necessary.

Defects and cost of late discovery

A defect is a mismatch between actual and expected behaviour. If a requirement is misunderstood early, correcting the wording may be inexpensive. If the same misunderstanding reaches production, the team may need code changes, data repair, customer communication, hotfixes and incident handling. The exact cost varies, but late defects generally involve more dependencies and disruption.

Shift left and shared responsibility

Shift left means bringing quality activities earlier: clarify acceptance criteria during refinement, review designs, run unit tests during development, and automate checks in CI. It does not mean eliminating system or acceptance testing. Developers, testers, product owners, designers and operations all contribute to quality.

Worked example

Example: A requirement says users may reset a password using a verified email address. Verification checks that the implementation follows the stated flow. Validation checks whether real users can understand and complete the reset safely.

Common mistakes

• Treating testing as only the tester's job.

• Confusing verification with validation.

• Assuming shift left means skipping later testing.

Knowledge check

Q: Verification asks what?
Answer: Whether the product matches its specification.

Q: Validation asks what?
Answer: Whether the product solves the user's real problem.

Q: What is shift left?
Answer: Doing quality work earlier in the delivery cycle.

Module 2 — Test Case Design Techniques

Systematic ways to get maximum coverage from minimum cases

Learning objectives

Use equivalence partitions, boundaries, decision tables and negative tests.

Equivalence partitioning

Divide inputs into groups expected to behave similarly, then choose representative values. If an age field accepts 18–60 inclusive, useful partitions are below 18 (invalid), 18–60 (valid), and above 60 (invalid). Testing every possible age is unnecessary if values in each partition truly follow the same rules.

Boundary value analysis

Defects often occur at range edges. For an inclusive valid range of 18–60, test 17, 18, 19, 59, 60 and 61. For a string length of 3–20, test lengths 2, 3, 4, 19, 20 and 21. Confirm whether boundaries are inclusive or exclusive from the requirement.

Decision tables

Use a table when outcomes depend on combinations of conditions. For a discount requiring both an active membership and an order above ■1,000, test all combinations: member/non-member and threshold met/not met. This exposes missing rules better than testing conditions individually.

Negative testing

Provide invalid, missing, malformed, unexpected or unauthorised input and confirm the system fails safely. Examples include an empty required field, invalid email, unsupported file type, expired session or malformed JSON. Verify a useful error, no crash, no unintended data change and no information leakage.

Worked example

Example age cases: 17 → reject; 18 → accept; 19 → accept; 60 → accept; 61 → reject. These cases target both partitions and boundaries.

Common mistakes

• Testing only typical values and ignoring edges.

• Using equivalence partitions without checking whether rules differ within a group.

• Testing each condition alone when the result depends on combinations.

Knowledge check

Q: For valid ages 18–60, name two boundary tests.
Answer: 17 and 18, or 60 and 61.

Q: When are decision tables useful?
Answer: When several conditions combine to determine an outcome.

Q: What is negative testing?
Answer: Checking that invalid or hostile inputs are rejected safely.

Module 3 — Writing Test Cases

Cases another tester can run and get the same result

Learning objectives

Write independent, repeatable test cases with precise expected results.

Test case structure

A strong test case includes an ID, title, requirement or feature reference, preconditions, test data, ordered steps, expected result, actual result, status, environment and relevant evidence. Not every team uses every field, but steps and expected results must be precise.

Preconditions and test data

A precondition describes the state needed before execution, such as “user account is active and verified.” Test data contains the exact inputs: a test account, a sample file, or a numeric value. Use controlled test data and avoid real customer information where possible.

Expected result and repeatability

Expected results should be observable and specific. “Works correctly” is vague; “the order confirmation page shows order number X and the order appears once in order history” is testable. Independent test cases should not rely on hidden state left by a previous case unless that dependency is documented.

Traceability

A requirements traceability matrix maps requirements to test cases and results. It helps teams see which requirements have no tests, which tests failed, and what needs rerunning after a change.

Worked example

ID: TC-LOGIN-04 Precondition: Registered active user exists. Steps: Enter valid email and password; select Sign in. Expected: User is authenticated and redirected to dashboard. Negative variant: Wrong password shows a safe error and does not create a session.

Common mistakes

• Writing expected results that cannot be observed.

• Leaving test data ambiguous.

• Creating cases that pass only because another test ran first.

Knowledge check

Q: What is a precondition?
Answer: The state required before a test can be run meaningfully.

Q: What makes an expected result good?
Answer: It is specific, observable and tied to the requirement.

Q: Why use traceability?
Answer: To show requirement coverage and support impact analysis.

Module 4 — Defect Reporting

Reports that get fixed instead of returned

Learning objectives

Report defects clearly and distinguish severity from priority.

A useful defect report

Include a concise title, environment/build, preconditions, exact reproduction steps, expected result, actual result, reproducibility, evidence such as logs/screenshots, and impact. Remove secrets and personal data from evidence.

Severity versus priority

Severity describes how badly the defect affects the system or users. Priority describes how urgently it should be fixed. A production payment failure may be high severity and high priority. A typo on a rarely used internal page may be low severity and low priority. A minor issue on a major campaign landing page may have low severity but high priority due to business timing.

Defect life cycle

A typical workflow is New → Triaged → Assigned → In Progress → Fixed → Ready for Retest → Verified/Closed. If the issue persists, reopen it. Teams may use different labels; the important part is clear ownership and status.

Retesting and regression

Retesting checks whether the reported defect is fixed using the failing scenario. Regression testing checks whether the change broke previously working behaviour elsewhere. A targeted retest is not a replacement for regression coverage.

Worked example

Title: Checkout fails for saved cards on mobile Environment: staging, Android browser, build 2.4.1 Steps: Sign in; add item; choose saved card; select Pay. Expected: Payment completes and one order is created. Actual: Error message appears; order is not created. Evidence: Sanitised console/network log.

Common mistakes

• Reporting “it is broken” without reproduction steps.

• Using severity and priority as synonyms.

• Closing a defect before retesting the fix.

Knowledge check

Q: What is severity?
Answer: How badly the defect affects functionality or users.

Q: What is priority?
Answer: How urgently it should be fixed.

Q: Retest versus regression?
Answer: Retest checks the fix; regression checks for unintended side effects.

Module 5 — Functional & Non-Functional Testing

Testing what it does and how well it does it

Learning objectives

Choose functional and non-functional test types based on risk.

Functional testing

Checks what the system does against requirements: login, checkout, search, permissions, calculations and error handling. Functional tests may be manual or automated and can run at several testing levels.

Smoke, sanity and regression

A smoke test is a broad, shallow check that a build is stable enough for deeper testing. A sanity test is a focused check around a particular change or fix. Regression testing reruns selected existing tests to find side effects. Teams sometimes use “sanity” differently, so agree on the local definition.

Non-functional testing

Performance, load, stress, scalability, reliability, usability, accessibility, compatibility and security tests assess quality attributes. Load testing measures behaviour under expected usage; stress testing explores behaviour beyond normal capacity. Usability testing observes whether representative users can complete tasks effectively.

Choosing tests by risk

A payment release deserves payment-path regression, failure and retry checks, authorisation tests, and load checks where traffic is a concern. A CSS-only change may need visual, responsive and accessibility checks. Select coverage based on what changed and the consequence of failure.

Worked example

Release example: Smoke: app opens and login works. Sanity: the fixed coupon calculation is correct. Regression: existing checkout, refunds and order history still work. Load: measure response time with expected concurrent users. Usability: observe whether a first-time user can find checkout.

Common mistakes

• Calling a smoke test exhaustive.

• Assuming performance testing only means load testing.

• Running the same large suite without considering change risk.

Knowledge check

Q: What is a smoke test?
Answer: A quick broad check that the build is stable enough for further testing.

Q: What does load testing measure?
Answer: Behaviour under expected usage volume.

Q: What does usability testing evaluate?
Answer: How easily users can complete intended tasks.

Module 6 — Levels of Testing

Unit, integration, system and acceptance

Learning objectives

Choose the correct testing level and apply the test pyramid.

Unit testing

Tests a small unit of logic in isolation, such as a tax calculation or password validation function. Unit tests are usually fast and numerous; dependencies may be replaced with controlled fakes or mocks.

Integration testing

Checks whether components work together across boundaries, such as an API handler with a database, a service with a payment provider sandbox, or two modules exchanging data.

System testing

Tests the integrated product against end-to-end system requirements. It can cover workflows such as sign-up, course enrolment, payment and certificate generation.

User acceptance testing (UAT)

Business representatives or intended users check that the product meets real acceptance needs and is fit for use. UAT is not simply a developer's unit test and should be based on agreed acceptance criteria.

Test pyramid

A useful strategy is many fast unit tests, a smaller number of integration tests, and a carefully selected set of end-to-end tests. UI tests are valuable but can be slower and more brittle. The ideal balance depends on the system.

Worked example

Example: Unit: calculate course completion percentage. Integration: save progress to the database. System: learner completes all modules and receives a certificate. UAT: a training coordinator confirms the workflow meets programme rules.

Common mistakes

• Testing everything only through the UI.

• Mocking every dependency and therefore missing integration failures.

• Treating UAT as a substitute for technical testing.

Knowledge check

Q: Which level tests one small function?
Answer: Unit testing.

Q: Which level checks components working together?
Answer: Integration testing.

Q: Who typically confirms business fitness in UAT?
Answer: Business representatives or intended users.

Module 7 — API Testing

Testing beneath the user interface

Learning objectives

Test API contracts, payloads, schemas and authorisation.

Endpoint contracts

A contract specifies method, path, authentication, required inputs, response schema, status codes and error behaviour. Contract tests catch changes that break consumers, such as renaming a field or changing a number to a string.

Payload and status checks

The payload is the request or response body. Verify status code, content type, required fields, data types, values and error format. Test missing fields, invalid types, oversized values and malformed JSON.

Authorisation testing

Authentication checks identity; authorisation checks permission. Test each role and resource boundary: a learner should not access another learner's private progress by changing an ID. Test unauthenticated requests, insufficient roles and access to another user's records.

Schema validation and chained workflows

Schema validation checks response structure against a formal definition. API workflow tests chain calls: create a resource, retrieve it, update it, verify the update and delete it. Use isolated test data and clean up afterward.

Worked example

Example: GET /api/courses/12 should return 200 and a JSON object with id as a number and title as a string. Without a valid session, a protected endpoint should return the agreed unauthenticated response. A learner must not retrieve another learner's private record.

Common mistakes

- Checking only status codes and ignoring payload contents.
- Testing authentication but not per-resource authorisation.
- Using production data or leaving test records behind.

Knowledge check

Q: What is a payload?
Answer: The data body in a request or response.

Q: What does a contract test protect?
Answer: The agreed structure and types consumers depend on.

Q: Why test changing a resource ID?
Answer: To verify that authorisation prevents access to other users' data.

Module 8 — Test Automation Fundamentals

Automating checks that earn their maintenance cost

Learning objectives

Choose useful automation and diagnose flaky tests.

What to automate

Automate stable, repeatable, high-value checks: critical workflows, calculations, API contracts, regression paths and data validation. Manual exploratory testing remains valuable for discovering unexpected behaviours and usability problems.

Assertions

An assertion states what must be true for a test to pass. Good assertions verify meaningful outcomes, not just that a page exists. For example, after checkout, assert that one order is created and the total is correct.

Flaky tests

A flaky test passes and fails intermittently without a relevant code change. Causes include timing assumptions, shared test data, test order dependence, network instability and nondeterministic dependencies. Prefer explicit waits for conditions, isolated data, deterministic setup and clear diagnostics instead of arbitrary sleeps.

Page Object Model

The Page Object Model wraps UI locators and actions in reusable page objects. Test intent stays readable while selectors are maintained in one place. Avoid turning page objects into huge classes that hide important test behaviour.

Maintainability and CI

Run fast tests frequently, isolate slow end-to-end suites where appropriate, preserve useful failure artifacts, and remove redundant tests. Automation has a maintenance cost; measure stability and value, not just the number of automated cases.

Worked example

Example pseudocode: openLoginPage() loginPage.signIn(validUser) assert dashboard.isVisible() assert dashboard.userName() == validUser.name

Common mistakes

• Using fixed sleeps for every asynchronous action.

• Sharing accounts or records across parallel tests.

• Automating unstable flows before understanding their expected behaviour.

Knowledge check

Q: What is an assertion?
Answer: A statement that defines what must be true for a test to pass.

Q: Name a common cause of flakiness.
Answer: Timing assumptions, shared state or unstable dependencies.

Q: What does Page Object Model centralise?
Answer: UI locators and page actions in reusable objects.

Module 9 — Security & Performance Awareness

Non-functional risks testers are expected to catch

Learning objectives

Interpret security and performance risks and communicate impact.

Input handling and injection risks

Validate input by type, length, range and allowed values. Prefer parameterised database queries; encode output for its context; sanitise rich HTML only when needed. Test malformed and unexpected input without using destructive payloads on systems you do not own or have permission to test.

Throughput and latency

Throughput is the number of successful requests or transactions handled per unit of time. Latency is the time from a request being made to its response. Average latency can hide slow outliers, so examine percentiles such as p95 or p99 alongside error rates and throughput.

Performance test interpretation

Measure under a defined workload and environment. Record concurrency, request mix, data volume, latency percentiles, errors and resource use. A service that responds quickly while dropping many requests is not healthy; throughput and error rate must be interpreted together.

Penetration testing and risk communication

A penetration test is an authorised, scoped simulated attack used to identify exploitable weaknesses. Confirm permission and scope before testing. Report the affected asset, conditions, evidence, impact, likelihood and recommended remediation in language business owners can use.

Worked example

Example business report: “During a 500-user load test, p95 checkout latency increased from 0.8 s to 4.2 s and 3% of requests failed. This may cause abandoned purchases during a campaign. Investigate database contention and repeat the test after remediation.”

Common mistakes

• Reporting latency without workload or percentile context.

• Testing security outside the authorised scope.

• Reporting technical findings without describing user or business impact.

Knowledge check

Q: What is throughput?
Answer: Successful requests or transactions per unit of time.

Q: What does latency measure?
Answer: Time between request and response.

Q: What must be true before a penetration test?
Answer: It must be authorised and within an agreed scope.

Module 10 — Testing in Agile Delivery

Quality as a continuous activity, not a final phase

Learning objectives

Work with acceptance criteria, Definition of Done, CI and exploratory testing.

Sprints and acceptance criteria

A sprint is a fixed period in which a team delivers a usable increment. Acceptance criteria make expected behaviour explicit. During refinement, testers help identify edge cases, missing rules, test data and non-functional risks before implementation.

Definition of Done

The Definition of Done is the team's shared checklist for considering work complete. It may require reviewed code, passing automated tests, acceptance criteria met, accessibility checks, updated documentation and no unresolved critical defects. It should be explicit and realistic.

Continuous integration

Continuous integration builds and tests changes frequently as they are integrated. A pipeline can run formatting, linting, unit tests, integration tests, security checks and build verification. Fast feedback makes failures easier to trace to a specific change.

Exploratory testing

Exploratory testing combines learning, test design and execution. The tester follows a charter such as “Explore enrolment and payment recovery on a slow connection,” records observations, and investigates unexpected behaviour. It is structured by a goal but not limited to a fixed script.

Quality reporting and collaboration

Report meaningful indicators: requirements covered, pass/fail/block status, open defects by risk, test environment limitations and release risks. Avoid reporting only a pass percentage without explaining the impact of critical failures. Quality is a team responsibility throughout the sprint.

Worked example

Example: Acceptance criterion — “A learner can enrol only once in a course.” During refinement, identify duplicate-click, retry and concurrent-request cases. During development, add unit/API checks. In CI, run regression tests. During review, report any unresolved duplicate-enrolment risk.

Common mistakes

• Treating testing as a separate final phase.

• Marking work done while acceptance criteria fail.

• Reporting a high pass percentage while hiding a critical blocker.

Knowledge check

Q: What does Definition of Done describe?
Answer: The shared conditions that must be met for work to count as complete.

Q: What is continuous integration?
Answer: Frequently integrating changes and automatically building/testing them.

Q: What distinguishes exploratory testing?
Answer: Learning, designing and executing tests together guided by tester judgement.

Practice Assessment — 20 Questions

Choose one answer per question. Target: 14/20 (70%). Answer first, then review the explanations.

1. Verification primarily checks whether:

A. The product matches its specification

B. The product is profitable

C. Users like the logo

D. DNS works

2. Shift left means:

A. Stop all testing before release

B. Move quality activities earlier

C. Test only in production

D. Remove code review

3. For a valid age range of 18–60 inclusive, which is a boundary test?

A. 35

B. 42

C. 17 and 18

D. 30 only

4. Decision tables are especially useful when:

A. Only one input exists

B. Multiple conditions combine to determine outcomes

C. No requirements exist

D. Testing fonts only

5. Which is a strong expected result?

A. It works

B. It looks fine

C. One order is created and a confirmation number is shown

D. No issues

6. Traceability maps:

A. Requirements to tests and results

B. CSS to images

C. Users to passwords

D. Servers to keyboards

7. Severity means:

A. How urgently a team wants the issue fixed

B. How badly the defect affects the system

C. Number of test cases

D. Age of the bug

8. Regression testing checks whether:

A. Previously working behaviour broke after a change

B. The user can spell

C. DNS records exist

D. The build has a name

9. A quick broad check that a build is stable enough for deeper testing is:

A. Penetration test

B. Smoke test

C. UAT only

D. Stress test

10. Testing response time under expected concurrent use is:

A. Load testing

B. Unit testing

C. Traceability

D. Verification only

11. Testing one isolated calculation is usually:

A. System testing

B. Unit testing

C. UAT

D. Exploratory testing only

12. A business user confirms the product meets acceptance needs during:

A. Unit testing

B. UAT

C. Linting

D. Merge conflict resolution

13. A contract test primarily checks:

A. Agreed API structure and types

B. UI colours only

C. User typing speed

D. Network cable length

14. A learner can change an ID in a URL to read another learner's record. This suggests a failure in:

A. Authorisation

B. CSS Grid

C. Boundary value analysis only

D. Page Object Model

15. A flaky test is one that:

A. Always fails with a known bug

B. Passes/fails intermittently without relevant code changes

C. Has no assertion

D. Runs only manually

16. Page Object Model centralises:

A. UI locators and page actions

B. Production passwords

C. DNS records

D. Database backups

17. Throughput is:

A. Time per request only

B. Successful requests per unit time

C. A defect priority

D. Password length

18. A penetration test should be:

A. Unauthorised and unlimited

B. Authorised and scoped

C. Run against any public site

D. Hidden from all stakeholders

19. The Definition of Done is:

A. A shared checklist for completion

B. A list of future features

C. A production password

D. A performance chart

20. Continuous integration typically:

A. Builds/tests changes frequently as integrated

B. Removes all testing

C. Means manual deployment only

D. Runs only after a major outage

Answer Key and Explanations

1. A — Verification checks conformance to specified requirements.

2. B — Early clarification and checks find defects when they are usually cheaper to fix.

3. C — 17 is just outside the lower boundary and 18 is the lower valid boundary.

4. B — They systematically cover combinations of conditions.

5. C — It is observable and specific.

6. A — Traceability helps show coverage and assess impact.

7. B — Severity is impact; priority is urgency.

8. A — Regression testing looks for unintended side effects.

9. B — Smoke testing is shallow and broad.

10. A — Load testing measures behaviour under expected load.

11. B — Unit tests focus on small units of logic.

12. B — UAT checks business/user fitness.

13. A — Contract tests protect consumers from incompatible API changes.

14. A — The server must enforce per-resource permissions.

15. B — Flakiness often arises from timing, shared state or nondeterminism.

16. A — Page objects help isolate UI details from test intent.

17. B — Throughput measures volume processed over time.

18. B — Security testing must have explicit permission and scope.

19. A — It describes agreed conditions for considering work complete.

20. A — CI gives rapid feedback on integrated changes.

Quick Revision Sheet

Quality: Quality is shared; verify against specification and validate against user needs. Shift left, but keep later test levels.

Test design: Use partitions, boundaries, decision tables and negative tests to target meaningful cases.

Test cases: Write clear preconditions, exact data, steps, observable expected results and requirement links.

Defects: Report reproducible steps and evidence. Severity is impact; priority is urgency. Retest fixes and run regression.

Test types: Smoke checks basic stability; regression checks side effects; load tests expected traffic; usability tests task completion.

Levels: Unit tests one piece; integration tests interfaces; system tests the product; UAT checks business fitness.

API: Check status, headers, payload, schema, error handling and role/resource permissions.

Automation: Automate stable, repeated, high-value checks. Use assertions, deterministic data and explicit waits to reduce flakiness.

Security/performance: Validate inputs, use authorised security tests, and interpret latency, throughput and error rates together.

Agile: Use acceptance criteria, Definition of Done, CI and exploratory testing throughout the sprint.

Final preparation tip

For each module, explain the concept, create one example test case, name a likely failure, and state what evidence would prove pass or fail. When reviewing MCQs, learn why the incorrect choices are wrong.